

Prompt Engineering

AI-Agent (Auto part Storage)



Groepsnaam: Fatneek

Docent: Dhr. Christian R. MS CS/ Mw. Bergwijn D.

Sem: 5.2

Groepsleden:

Ramdhiansing Shakeel	SE/1123/067
Ramkhelawan Riaaz	SE/1123/070
Sangham Rishika	SE/1123/081
Sariman Jalen	SE/1123/082
Sodipo Sherreskly	SE/1123/086

Inhoudsopgave

Inleiding	4
1. Beschrijving van de Use Case	5
Korte beschrijving van het probleem:	5
Doel van de AI-ondersteuning:.....	5
Rol van Prompt Engineering binnen deze oplossing:	5
2. Analyse van het Prompt Ecosysteem	6
Gebruiker:.....	6
Context.....	6
Model	6
Output.....	7
Prompt Ecosysteem	8
3. Prompttypes en Ontwerpkeuzes	10
4. Iteratie en Verbetering (Prompt Cyclus)	13
System Prompt – Role-based Prompt.....	13
User Prompt – Query Prompt	14
5. Kwaliteitscriteria	16
Effectiviteit.....	16
Efficiëntie	16
Consistentie van Output	17
Foutgevoeligheid en Randgevallen.....	18
Conclusie.....	19

Lijst met figuren

Figuur 1: Prompt Ecosysteem	8
Figuur 2: Role-based Prompt	10
Figuur 3: Instruction Prompt	11
Figuur 4: Contextual Prompt	12
Figuur 5: Vb. Slechte User Prompt.....	14
Figuur 6: Vb. Goeie User Prompt	15

Figuur 7: Vb. Intent & Entiteiten Extractie	16
Figuur 8: Vb. Databasecategorisatie Entiteiten	16
Figuur 9: Vb. General Gebruikersvraag.....	17

Inleiding

De snelle ontwikkeling van kunstmatige intelligentie heeft nieuwe mogelijkheden gecreëerd voor het verbeteren van zoek- en interactiesystemen binnen digitale applicaties. Een technologie die hierbij een belangrijke rol speelt is het gebruik van Large Language Models (LLM's). Deze modellen maken het mogelijk om natuurlijke taal te interpreteren en contextuele antwoorden te genereren op basis van gebruikersvragen.

In traditionele systemen moeten gebruikers vaak zoeken via vaste filters of exacte productnamen. Dit kan problematisch zijn in situaties waarin gebruikers niet precies weten welk product zij nodig hebben of wanneer zij niet bekend zijn met technische productcodes. Dit probleem komt vaak voor bij auto-onderdelen, waar compatibiliteit afhankelijk is van specifieke voertuigkenmerken zoals merk, model en bouwjaar.

Binnen dit project is een AI-chatbot ontwikkeld die gebruikers ondersteunt bij het zoeken naar auto-onderdelen. De chatbot stelt gebruikers in staat om vragen te stellen in natuurlijke taal, waarna het systeem relevante productinformatie uit een database kan ophalen en een begrijpelijk antwoord kan genereren.

Een cruciaal onderdeel van dit systeem is **Prompt Engineering**. Het verwijst naar het proces van het ontwerpen en optimaliseren van instructies die aan een taalmodel worden gegeven om de gewenste output te verkrijgen. Door effectieve prompts te ontwerpen kan het model beter worden gestuurd om correcte, relevante en consistente antwoorden te genereren.

In dit verslag wordt het ontwerp en de optimalisatie van prompts beschreven voor de AI-chatbot binnen het auto-onderdelen systeem. Hierbij wordt aandacht besteed aan het prompt ecosysteem, verschillende prompttypes, iteraties van prompts en de evaluatie van de kwaliteit van de gegenereerde antwoorden.

1. Beschrijving van de Use Case

Korte beschrijving van het probleem:

Auto-onderdelen winkels beschikken vaak over grote productcatalogi met duizenden verschillende onderdelen. Deze onderdelen verschillen per categorie, maar ook per automerk, model, bouwjaar en motorvariant. Daarnaast moet rekening worden gehouden met compatibiliteit tussen onderdelen en voertuigen, evenals met actuele voorraadniveaus.

Voor veel klanten is het daardoor moeilijk om het juiste onderdeel te vinden. Klanten beschikken vaak niet over voldoende technische kennis van auto-onderdelen en weten meestal niet welke specifieke productnaam, SKU of part-nummer nodig is voor hun voertuig. Hierdoor wordt het zoeken naar een geschikt onderdeel complex en tijdrovend.

Traditionele zoekinterfaces vereisen vaak dat gebruikers exacte productnamen of onderdeelcodes invoeren. Wanneer deze informatie niet bekend is, wordt het vinden van het juiste product lastig. Dit kan leiden tot frustratie bij klanten, een grotere kans op verkeerde aankopen en extra werk voor medewerkers die klanten handmatig moeten helpen.

Doel van de AI-ondersteuning:

Het doel van de AI-oplossing is het ontwikkelen van een chatbot die gebruikers helpt bij het zoeken van auto-onderdelen en eventueel reservering maken door natuurlijke taal te gebruiken. De gebruiker kan bijvoorbeeld vragen:

- *“Welke remblokken passen op een Toyota Corolla 2015?”*
- *“Hebben jullie een luchtfilter voor Honda Civic?”*

De AI-chatbot interpreteert de vraag, zoekt relevante informatie in de database en genereert een begrijpelijk antwoord voor de gebruiker.

Rol van Prompt Engineering binnen deze oplossing:

Prompt engineering speelt een belangrijke rol bij het sturen van het taalmodel, zodat:

- Gebruikersvragen correct worden geïnterpreteerd
- Alleen relevante productinformatie uit de database wordt opgehaald en gebruikt
- Antwoorden duidelijk en consistent worden geformuleerd
- Foutieve of hallucinerende antwoorden worden voorkomen

Goed ontworpen prompts helpen het model om de juiste context te gebruiken en betrouwbare antwoorden te genereren.

2. Analyse van het Prompt Ecosysteem

Het prompt ecosysteem bestaat uit verschillende componenten die samen bepalen hoe effectief de AI-output is.

Gebruiker:

De gebruiker van het systeem is een klant of medewerker van een auto-onderdelen winkel.

Kenmerken van de gebruiker:

- Beperkt technische kennis van auto-onderdelen
- Gebruikt natuurlijke taal
- Kent vaak geen exacte product codes

Voorbeelden van gebruikersvragen:

- *“Ik zoek een olie filter voor een Toyota Hilux”*
- *“Welke onderdelen hebben jullie voor een Hyundai SantaFe?”*

Het prompt ontwerp moet dus rekening houden met **variërende formuleringen en beperkte technische kennis van gebruikers**.

Context

Het taalmodel heeft context nodig om correcte antwoorden te genereren.

De context kan bestaan uit:

- Productinformatie uit de database
- Compatibiliteit tussen producten en automodellen
- Samenvatting van eerdere chatberichten
- Intent van vraag
- Entiteiten van vraag (vb. bouwjaar: *2012*, auto_merk: *“Toyota”*)
- Prompts nl. Developer-, System- en User prompts

Deze informatie wordt aan het model meegegeven, zodat het antwoord gebaseerd is op echte data en het betrouwbare en consistente data kan genereren.

Model

Het systeem maakt gebruik van een externe **Large Language Model (LLM)** via een API van **Groq**.

Model naam: **“Llama-3.3-70B-Versatile”**

Het model wordt gebruikt om:

- Gebruikersvragen te interpreteren (bijv. intent en entiteiten bepalen)
- Database-informatie te verwerken
- Natuurlijke antwoorden te genereren
- Extractie van informatie uit user prompts
- Intentie van gebruikers te bepalen
- Samenvattingen te genereren voor toekomstige context (in JSON-formaat)
- Context uit eerdere berichten in een gesprek te begrijpen
- Ontbrekende informatie te identificeren en vervolgvragen te stellen
- Gebruikers te begeleiden bij zoekopdrachten en reserveringen
- Irrelevante of ongeldige verzoeken te herkennen en te filteren

Het model fungeert dus als een interface tussen menselijke taal en gestructureerde databasegegevens.

Output

De gewenste output van het systeem is:

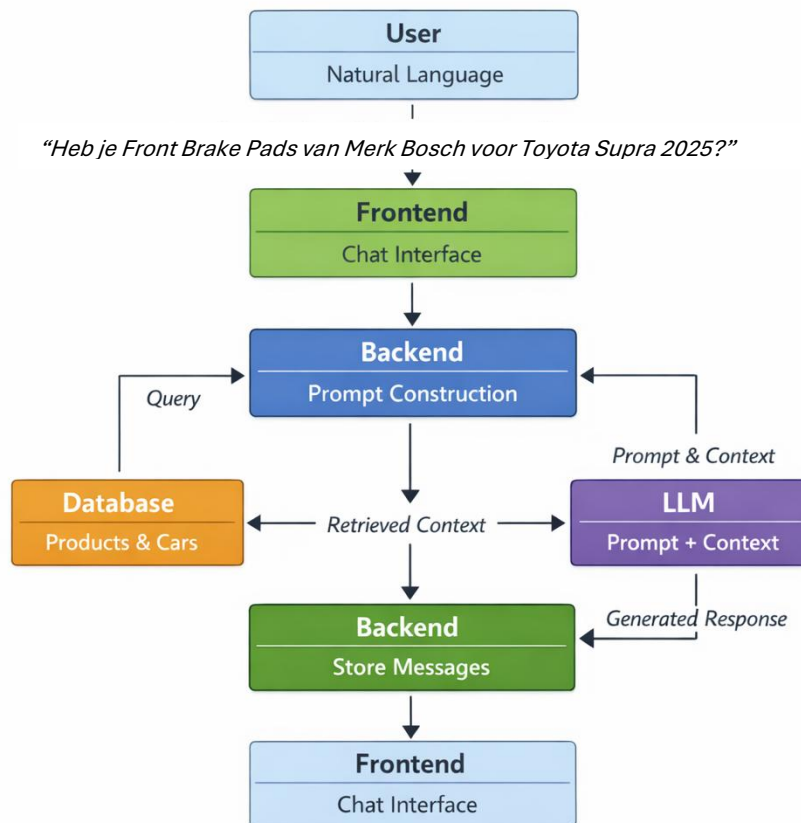
- Correct
- Relevant
- Gebaseerd op database-informatie
- Begrijpelijk voor de gemiddelde gebruiker
- Binnen de Autoparts Shop scope

Voorbeeld van gewenste output:

*“De volgende remblokken zijn compatibel met een Toyota Corolla 2015:
Bosch Brake Pad BP203 (voorraad: 12 stuks).”*

Het antwoord moet informatief zijn zonder onnodige technische complexiteit.

Prompt Ecosysteem



Figuur 1: Prompt Ecosysteem

Uitleg:

- **Gebruiker:** stelt vraag in natuurlijke taal via de chatinterface. Deze vraag vormt de initiële user prompt.
 - Voorbeeld: "“Heb je Front Brake Pads van Merk Bosch voor Toyota Supra 2025?”"
- **Frontend:** systeem (de chatinterface) stuurt de vraag naar de backend via een API-request.
- **Backend:** verwerkt de gebruikersvraag en bouwt de prompt die naar het LLM wordt gestuurd.
 - Het model ontvangt:
 - Gebruikersvraag
 - Context (geschiedenis en samenvatting → Contextual Prompt)
 - Developer Instruction Prompts

- Resultaat: Intent en Entiteiten van gebruikersvraag
- **Database:** De backend haalt relevante informatie op uit de database o.b.v. intent en entiteiten
 - Zoals:
 - Productnamen
 - Compatibele automodellen
 - Voorraadgegevens enz.
 - Resultaat: Databasecontext
- **LLM:** data wordt weer gestuurd naar de LLM om een antwoord te genereren.
 - Het taalmodel ontvangt:
 - De prompts (User & System)
 - De databasecontext
 - Overige context (geschiedenis en samenvatting → Contextual Prompt)
 - Resultaat: Gegeneerde antwoord
- **Response naar gebruiker:** de gegenereerde response wordt via de backend teruggestuurd naar de frontend en weergegeven aan de gebruiker.
 - Antwoord: *“Ik heb iets gevonden voor jou! We hebben een set voorste remschoenen van Bosch, specifiek voor de Toyota Supra uit 2025, met part nummer FBP-TOYOTA-SUPRA-2025, voor €55. We hebben er 20 in voorraad. Wil je hier meer over weten of direct bestellen?”*

Waarom dit ecosysteem belangrijk is

Het prompt ecosysteem laat zien dat AI-antwoorden niet alleen afhankelijk zijn van het taalmodel, maar ook van:

- De instructies in de systeemprompt en developer prompt
- De kwaliteit van de gebruikersprompt
- De context uit de database

3. Prompttypes en Ontwerpkeuzes

Voor de AI-oplossing zijn verschillende prompttypes ontworpen.

Prompt 1: System Prompt

Doel: Het doel van deze prompt is om het model een specifieke rol te geven, namelijk die van een behulpzame AI-assistent voor een autopart shop chatbot.

Type prompt: Role-based Prompt

Prompt:

```
function buildPrompt(body: ChatRequestBody) {
  const system = `
Je bent AutoBot, een behulpzame AI-assistent voor een autopart shop.

Je helpt alleen met:
- auto-onderdelen
- auto-onderhoud
- auto-problemen
- eenvoudige productgerichte vragen

Regels:
- Antwoord kort, duidelijk en praktisch
- Blijf binnen de context van auto's en auto-onderdelen
- Als de vraag buiten scope is, stuur de gebruiker netjes terug naar auto-gerelateerde onderwerpen
- Verzin geen voorraad, prijs of beschikbaarheid als je die niet zeker weet
- Geef geen onnodig lange uitleg
`.trim();
}
```

Figuur 2: Role-based Prompt

Verwachte output:

Een antwoord dat:

- Kort en praktisch is
- Alleen over auto-onderdelen gaat (binnen de scope)
- Geen informatie verzint
- Begrijpelijk is voor gebruikers

Onderbouwing van ontwerpkeuzes:

Door het model een rol te geven wordt het antwoord vaak consistent en relevanter voor de use case.

Prompt 2: Developer Instruction Prompt

Doel: Het doel van deze prompt is om het model instructies te geven over wat het met de gebruikersvraag moet doen, wat het moet interpreteren eruit en o.b.v. wat (regels).

Type Prompt: Instruction Prompt/ Policy Prompt

Prompt:

```
instructions: [
  "Bepaal ALLEEN de intent van de gebruiker.",
  "Antwoord ALLEEN met geldige JSON.",
  "Gebruik exact het output schema.",
  "Gebruik de user message als hoofdbron.",
  "Gebruik chat summary en recente history alleen als context bij korte of onduidelijke berichten.",
]
```

Figuur 3: Instruction Prompt

Verwachte output:

Een antwoord dat:

- In JSON-format is
- Gebaseerd is op gebruikersvraag en/of chat summary
- Duidelijk en beknopt is
- Juiste intent interpreteert

Onderbouwing van ontwerpkeuzes:

De instructie beperkt het model tot het alleen uitvoeren van een specifieke taak met vastgestelde regels. Dit zorgt voor efficiëntie door onnodige informatie te vermijden.

Prompt 3: Contextual Prompt

Doel: Het doel van deze prompt is om het model extra context te geven over het lopende gesprek, zodat het de intentie van de gebruiker nauwkeuriger kan bepalen.

Type prompt: Contextual Prompt

Prompt:

```
const user =  
User message:  
${(body.message || "").trim()}  
  
Chat summary:  
${body.chatSummary || "(none)" }  
  
Recent history:  
${history || "(none)" }
```

Figuur 4: Contextual Prompt

Verwachte output:

- De contextual prompt beïnvloedt niet het formaat van de output, maar helpt het model om een **nauwkeurigere classificaties** te maken.
- Helpt verder bij het genereren van correcte antwoorden wegens genoeg context.

Onderbouwing van ontwerpkeuzes:

- In natuurlijke gesprekken verwijzen gebruikers namelijk vaak naar eerdere berichten zonder alle informatie opnieuw te herhalen.
- Verminderen van verkeerde classificaties
 - Met chatgeschiedenis kan het model herkennen dat dit aanvullende informatie is binnen een lopende zoekvraag.
- Verbetering intent detective
 - Door genoeg context kan het model beter begrijpen wat de gebruiker bedoelt.

4. Iteratie en Verbetering (Prompt Cyclus)

System Prompt – Role-based Prompt

Iteratie 1: vage prompt

Prompt: *‘Beantwoord de gebruikersvragen door gebruik te maken van productinformatie.’*

Output (samengevat):

- Incomplete antwoorden
- Verzonnen producten
- Antwoorden zonder duidelijke structuur.

Analyse:

- Te weinig instructies
- Model wist niet dat het alleen database data moest gebruiken
- Antwoordstructuur was inconsistent.

Iteratie 2: prompt met rol en structuur

Prompt: *‘Je bent AutoBot, een behulpzame AI-assistent voor een autopart shop.’*

Je helpt alleen met:

- *auto-onderdelen*
- *auto-onderhoud*
- *autoproblemen*
- *eenvoudige productgerichte vragen*

Regels:

- *Antwoord kort, duidelijk en praktisch*
- *Blijf binnen de context van auto's en auto-onderdelen*
- *Als de vraag buiten scope is, stuur de gebruiker netjes terug naar auto-gerelateerde onderwerpen*
- *Verzin geen voorraad, prijs of beschikbaarheid als je die niet zeker weet*
- *Geef geen onnodig lange uitleg`*

Verbeterde Output (samengevat)

- Consistenter
- Duidelijker
- Betrouwbaarder

User Prompt – Query Prompt

Iteratie 1: Slechte Prompt

Prompt: “Heb je auto-onderdelen?”

Output (samengevat):

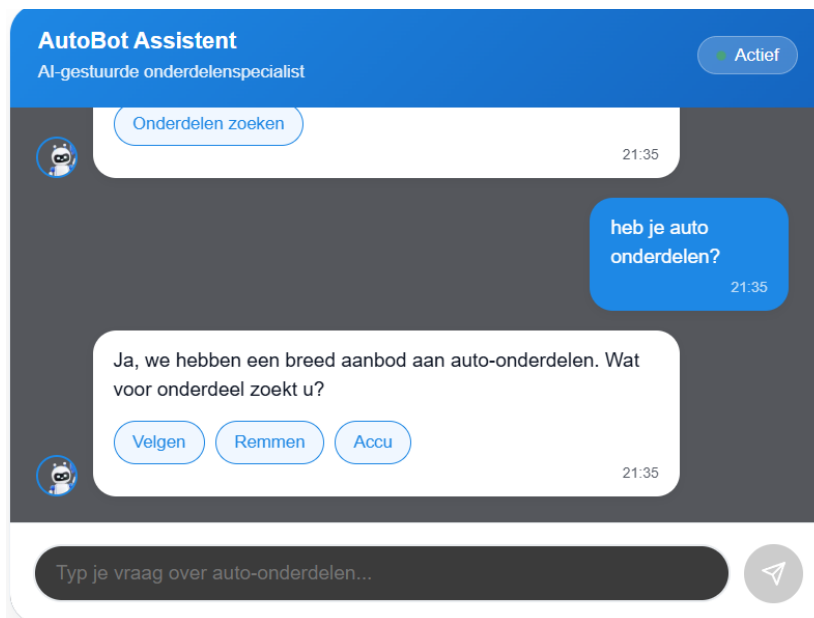
- Gebruiker moet meerdere vervolgvragen stellen, voordat hij juiste product vindt
- Onvoldoende context
- Algemene antwoord

Analyse:

Deze prompt is problematisch omdat:

- Vraag te algemeen is
- Geen voertuig informatie wordt gegeven
- Geen specifiek onderdeel wordt genoemd
- Gebruiker lang bezig zal zijn te chatten, voordat hij/zij het correcte product vindt.

Voorbeeld:



Iteratie 2: Goede Prompt

Prompt: “Heb je Front Brake Pads van Merk Bosch voor Toyota Supra 2025?”

Figuur 5: Vb. Slechte User Prompt

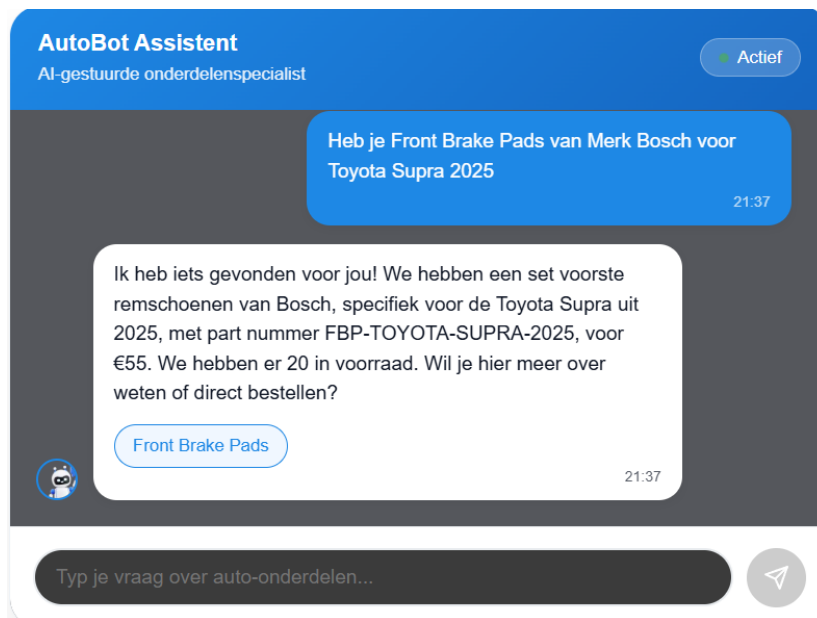
Verbeterde Output (samengevat):

- Correcte en voldoende context
 - Specifiek onderdeel (front brake pads)
 - Automerk (Toyota)
 - Model (Supra)
 - Bouwjaar (2025)

Analyse:

Beide prompts zullen werken, maar een prompt met voldoende context is effectiever voor het correct interpreteren van het probleem en op halen van juiste data.

Voorbeeld:



Figuur 6: Vb. Goeie User Prompt

5. Kwaliteitscriteria

De prompts zijn beoordeeld op verschillende kwaliteitscriteria.

Effectiviteit

De prompts slagen erin om relevante productinformatie uit de database te verwerken en gebruikersvragen correct te beantwoorden.

Efficiëntie

De prompts bevatten alleen noodzakelijke instructies en context. Onnodige informatie wordt vermeden om tokengebruik te minimaliseren.

Voorbeeld Effectiviteit + Efficiëntie:

- Gebruikersvraag: “*Heb je front brake pads van merk Bosch voor Toyota rav4 2020?*”
- LLM Interpretatie (gedeeltelijk):

```
"intent": "SEARCH",
"searchState": {
  "part": "front brake pads",
  "brand": "bosch",
  "autoBrand": "toyota",
  "model": "rav4",
  "year": "2020",
  "missingFields": [],
  "readyForDbSearch": true,
  "lastAskedField": ""
```

Figuur 7: Vb. Intent & Entiteiten Extractie

```
"productId": 93,
"productName": "Front Brake Pads",
"productBrand": "Bosch",
"categoryName": "Brake Parts",
"partNumber": "FBP-TOYOTA-RAV4-2020",
"sku": "SKU-FBP-TOYOTA-RAV4-2020",
"salePrice": 55,
"stockQuantity": 20,
"autoBrand": "Toyota",
"autoModel": "RAV4",
"year": 2020,
"compatibilityId": 205
```

Figuur 8: Vb. Databasecategorisatie Entiteiten

- Chatbot Antwoord: “*Ik kon geen duidelijke match vinden voor spark plug voor een Hilux 2023. Ik kan wel verder zoeken op alternatief onderdeel of part number.*”

Uitleg:

- Gebruiker stelt vraag
- Model interpreteert vraag
 - Haalt intent en entiteiten uit vraag (Instruction Prompt)
- Als intent == Search → zoek in database
- LLM formuleert antwoord o.b.v. query resultaten en blijft binnen de autopart scope.

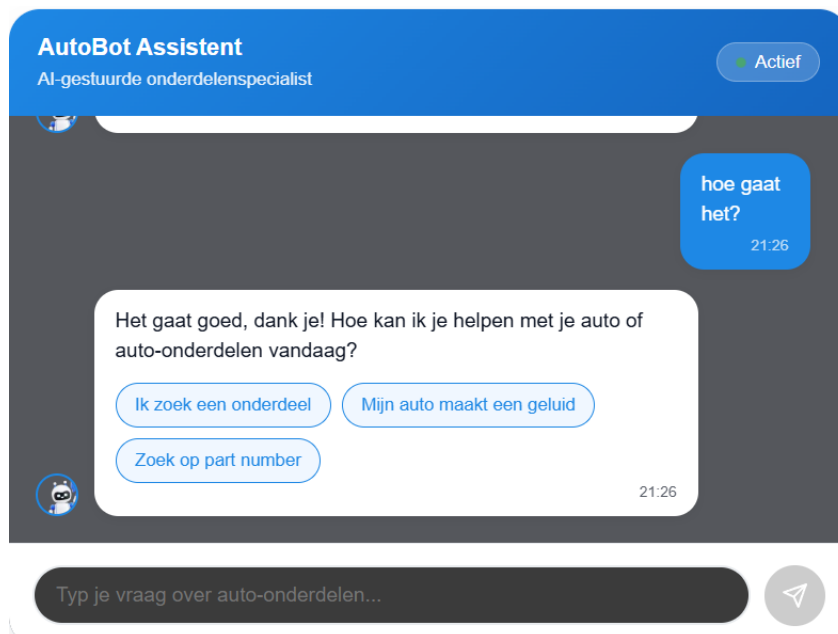
Consistentie van Output

Door gebruik van rolprompts en contextual prompting wordt de output consistentier qua structuur en formulering.

Voorbeeld:

- Gebruikersvraag: “Hoe gaat het?”
- Chatbot antwoord: “Het gaat goed, dank je! Hoe kan ik je helpen met je auto of auto-onderdelen vandaag?”

Voorbeeld:



Figuur 9: Vb. General Gebruikersvraag

Uitleg:

- Gebruiker stelt vraag
- Model haalt intent == General
- LLM geeft antwoordt, maar blijft binnen de context van zijn rol (AI-Assistent binnen een autopart shop) (Role-based & Contextual Prompt)

Foutgevoeligheid en Randgevallen

Mogelijke randgevallen zijn:

- Gebruikers vragen naar niet-bestaande producten
- Gebruikers geven onvolledige auto-informatie
- Meerdere compatibele producten

De prompts instrueren het model om in deze gevallen duidelijk aan te geven wanneer informatie ontbreekt.

Conclusie

In dit project werd prompt engineering toegepast om een AI-chatbot te ontwikkelen die gebruikers helpt bij het vinden van auto-onderdelen via natuurlijke taal. Door middel van zorgvuldig ontworpen prompts kan het taalmodel gebruikersvragen interpreteren en relevante productinformatie uit de database verwerken in begrijpelijke antwoorden.

Uit het ontwikkelproces bleek dat de kwaliteit van de output sterk afhankelijk is van de manier waarop prompts worden geformuleerd. Algemene prompts leiden vaak tot onduidelijke of onnauwkeurige antwoorden. Door prompts te voorzien van duidelijke instructies, contextinformatie en voorbeeldinteracties kan het gedrag van het taalmodel aanzienlijk worden verbeterd.

Het iteratieve ontwerpproces speelde een belangrijke rol bij het optimaliseren van de prompts. Door verschillende versies van prompts te testen en te analyseren konden problemen zoals hallucinerende antwoorden en inconsistente output worden verminderd. Het toevoegen van rolgebaseerde instructies en contextueel prompting bleek bijzonder effectief bij het verbeteren van de consistentie en bruikbaarheid van de antwoorden.

Daarnaast werd duidelijk dat prompt engineering niet alleen afhankelijk is van de prompt zelf, maar ook van de bredere architectuur van het systeem. De combinatie van databasecontext, backend verwerking en het taalmodel vormt samen het prompt ecosysteem dat bepaalt hoe effectief de AI-oplossing functioneert.

De resultaten van dit project laten zien dat goed ontworpen prompts een essentieel onderdeel vormen van AI-gedreven applicaties. Door een gestructureerde aanpak van prompt engineering toe te passen kan de betrouwbaarheid, relevantie en bruikbaarheid van AI-gegenereerde antwoorden aanzienlijk worden verbeterd.